
Guion de la defensa

Caso 2 — Línea de producción con balanceo dinámico

Estructura de Computadores y Sistemas Distribuidos · Evaluación 3

César Olivares · Simón García-Huidobro · 26 de agosto de 2026

Guion hablado para las 22 láminas de `presentacion_caso2.pdf`, una sección por diapositiva. Está escrito para decirse en voz alta, no para leerse en pantalla: en cada lámina el texto dice qué usamos, cómo lo usamos, por qué, y qué aporta a lo que queremos demostrar.

Duración. El guion tiene ~2.100 palabras habladas: unos **15 minutos** a ritmo normal (145 palabras por minuto), más **~4 de demo**. Si el bloque disponible es menor, recortar en este orden, y decidirlo en el ensayo, no en la defensa:

- Cortar los párrafos marcados con **<opcional>** .
- **Acortar la demo:** el paso 5 (detener Redis) se puede contar en vez de hacerse; su evidencia está en los anexos del informe.
- **Saltarse la lámina 20** (límites conocidos): de la 19 se pasa directo a conclusiones sin romper el hilo.

La regla es cortar contenido, nunca hablar más rápido.

Turnos. Cortes naturales: láminas 1–6 (problema y arquitectura), 7–11 (balanceo, carrera y detector), demo, 13–17 (experimentos), 18–22 (robustez y cierre). Definir el reparto antes y anotarlo acá: _____

DIAPPOSITIVA 1 Portada

“Hola, profesor. Presentamos el Caso 2: una línea de producción con balanceo dinámico de carga. Construimos un sistema distribuido que simula una planta conservera, detecta en vivo dónde se está atascando la producción, y responde una pregunta concreta: qué pasa con ese atasco cuando se agregan máquinas a la etapa lenta.”

DIAPPOSITIVA 2 El problema

“Antes del caso, el problema general. En toda línea en serie el ritmo lo fija la etapa más lenta, y el trabajo se acumula en fila frente a ella. A la izquierda, una línea sin nuestro desarrollo: la fila crece sin límite y sin aviso. A la derecha, lo que construimos: un detector que mide la fila de cada etapa, y capacidad agregada exactamente donde la fila crece.

Esos son los dos problemas de ingeniería del proyecto: repartir el trabajo entre copias de la etapa lenta sin duplicar ni perder unidades, y determinar midiendo cuál etapa limita la producción. Van juntos: replicar sin medir es invertir a ciegas, y medir sin poder replicar es diagnosticar sin tratamiento.”

DIAPPOSITIVA 3 El caso

“El caso concreto: una planta conservera ficticia de jurel en lata, con cuatro etapas en serie. La unidad de trabajo es el **lote**; cada etapa demora un tiempo fijo por lote —su **tiempo de ciclo**: cuatro, doce, tres y siete segundos— y entre etapas hay una **cola**: la fila de lotes esperando. Si a una etapa le llega trabajo más rápido de lo que procesa, su fila crece: eso es un **cuello de botella**.

Envasado, con doce segundos, es la más lenta, y es la que se replica. La pregunta del caso: ¿qué pasa con el cuello al escalar la etapa lenta? Adelanto la respuesta, porque todo lo que sigue es su evidencia: **no desaparece — se muda.**”

DIAPPOSITIVA 4 Tecnologías y su rol

“Las piezas y su rol; todo lo que digamos después sale de esta tabla.

Los servicios están escritos en **Python**. **Docker** empaqueta cada programa en una **imagen**; cada copia corriendo es un **contenedor**, y replicar una etapa es lanzar más contenedores de la misma imagen. **Redis** es un almacén en memoria que usamos como cola de trabajo; atiende sus comandos de a uno — ese detalle será clave en la condición de carrera. Con **FastAPI** construimos las dos **APIs**, la interfaz por la que un programa recibe peticiones de otro: la de órdenes y la de métricas. **SQLite** guarda órdenes y eventos en un archivo. Y **Streamlit** es el

tablero del operador.”

DIAPPOSITIVA 5 Arquitectura en nodos

“Un **nodo lógico** es un grupo de contenedores que correría en una máquina propia. Usamos tres, con un criterio: cada nodo escala o falla por un motivo distinto. El A atiende al operador; el B tiene las estaciones, lo único que se multiplica bajo carga; el C concentra el estado compartido.

Como la frontera entre nodos es la frontera entre contenedores, hoy conviven en una máquina y mañana se separan sin cambiar código. Y separar nodos separa dominios de falla: si las estaciones saturan el procesador, el operador sigue atendido.”

DIAPPOSITIVA 6 Una imagen, cuatro estaciones

“Las cuatro estaciones son un solo programa: una sola imagen de Docker. Al arrancar, cada contenedor recibe por configuración qué etapa es, de qué cola lee, a cuál escribe y cuánto dura su ciclo.

¿Por qué replicar? Envasado procesa un lote cada doce segundos, menos de lo que le llega: la única forma de subir su capacidad es poner más envasadoras en paralelo. Y como todas salen de la misma imagen, eso es un solo comando, sin tocar configuración. Ninguna réplica sabe cuántas hermanas tiene, ni lo necesita.”

DIAPPOSITIVA 7 Reparto por turnos: round-robin

“¿Cómo repartir el trabajo entre las réplicas? Primero la forma clásica, que es el punto de comparación.

Round-robin: asignar por turnos fijos — el lote uno a la réplica uno, el dos a la dos, y vuelta a empezar. Es lo que hace un balanceador por defecto: simple, y parejo en cantidad.

Su límite: es ciego al estado de cada réplica. Si una está ocupada, lenta o caída, igual le toca su turno — y el trabajo se apila frente a ella.”

DIAPPOSITIVA 8 Balanceo por demanda

“Nuestro desarrollo hace lo contrario: nadie asigna. Cada réplica **pide** el siguiente lote de la cola compartida cuando queda libre; el balanceador es la cola.

¿Cómo ayuda? Una réplica lenta simplemente pide menos veces: el reparto se adapta solo a la capacidad real de cada máquina — balanceo **dinámico**, sin programarlo. Y agregar réplicas

no requiere reconfigurar nada: basta conectarse a la cola. El experimento que lo mide viene más adelante: once, doce y siete.”

DIAPPOSITIVA 9 Coordinación: la condición de carrera

“Pedir de una cola compartida abre el problema central: si dos réplicas piden a la vez, el lote debe llevárselo exactamente una. Eso es una **condición de carrera**, y la provocamos antes de resolverla; ambas versiones están en el historial de commits.

La versión con defecto reclama en dos pasos: mira la cola, y después saca. Entre un paso y otro, otra réplica ve el mismo lote: con tres réplicas y doscientas órdenes medimos órdenes procesadas dos veces.

La solución es BRPOP, un comando de Redis que entrega el elemento y lo elimina en **una sola operación indivisible**. Como Redis atiende de a uno, dos réplicas no pueden recibir el mismo lote. Misma prueba: cero duplicados. Un candado habría *administrado* la ventana de riesgo; la operación atómica la *elimina*.”

DIAPPOSITIVA 10 Criterio de cuello: espera, no servicio

“Para localizar el cuello hay que decidir qué medir; esta es nuestra decisión técnica central.

De los eventos derivamos dos tiempos por etapa: la **espera** —desde que el lote entra a la cola hasta que lo toman— y el **servicio** —lo que dura el procesamiento—. Un servicio largo solo dice que la tarea es larga; el atasco es trabajo llegando más rápido de lo que se drena, y eso se ve en la espera.

El gráfico lo demuestra: con dos réplicas, envasado sigue con el ciclo más largo, pero la fila que crece está en esterilización. Por servicio, jamás la habríamos encontrado.”

DIAPPOSITIVA 11 El detector en funcionamiento

“La regla que corre en la API de métricas — el diagrama es esa lógica. *Advertencia* si la espera promedio del último minuto supera una vez y media el ciclo de la etapa; *crítico* si supera tres veces. Umbrales relativos, porque diez segundos de espera son graves para sellado y poca cosa para envasado. El cuello es la etapa de mayor espera entre las que superan umbral; si ninguna, no hay cuello.

Así lo evidenciamos: a ritmo constante, el detector señaló envasado. Agregamos una envasadora: la espera bajó, pero la fila reapareció en esterilización. Con una tercera, envasado quedó sobrado — y el límite siguió ahí. Los números vienen en los resultados.”

DIAPPOSITIVA 12 Demo en vivo (~4 minutos)

“Vamos a mostrarlo funcionando. Esto corre de verdad: dos réplicas de envasado y las otras tres etapas.”

Guion de la demo:

1. **La línea al día.** Mostrar las cuatro etapas, las réplicas con su ciclo y su carga, y el cartel en estado normal.
2. **Ingresar lotes** desde el panel izquierdo, **espaciados unos cinco segundos**.

Ojo: no inyectar muchos de golpe. Se apilan en la primera cola y el tablero marca fileteado como el atasco — correctamente, pero es la respuesta a otra pregunta. Si alguien lo nota, explicar exactamente eso.

3. **Ver crecer la espera** de esterilización. El cartel pasa de gris a ámbar y luego a rojo.

“El cartel no dice solo ‘hay un problema’: nombra la etapa, dice cuántos lotes hacen cola y da el número. Ese diagnóstico viene de la API de métricas; el tablero solo lo muestra.”

4. **La condición de carrera, en vivo.** Bajar y apretar «Ejecutar la comparación».

*“Esto corre el experimento ahora: encola los mismos lotes dos veces, una con cada versión del reclamo. Con cien pedidos, la versión en dos pasos registra unos trescientos envasados y la atómica exactamente cien. Y es **el mismo código** que corre en las estaciones: las dos funciones se importan del mismo módulo. Usa una cola aparte, así que la línea de arriba sigue produciendo.”*

5. **Detener Redis.** Parar el contenedor, mostrar que la interfaz nombra al servicio caído sin un error críptico, y volver a levantarlo.

Plan B si algo falla: las capturas están en los anexos del informe.

DIAPPOSITIVA 13 Experimentos: qué corrimos y por qué

“Los experimentos son software: un script crea una orden cada cinco segundos durante tres minutos a través de la API, y al final consulta las métricas. Se repite con una, dos y tres envasadoras; la corrida con una es la línea **sin** nuestro desarrollo — la base de comparación. Ritmo constante porque representa demanda sostenida; inyectar de golpe responde otra pregunta y va aparte.

El gráfico anticipa el resultado: cada barra es la capacidad de una etapa; la línea segmentada, los doce lotes por minuto que llegan. Toda etapa bajo esa línea acumula fila: envasado con una réplica saca cinco, con dos sube a diez — aún insuficiente — y esterilización, con ocho coma seis, queda como el siguiente límite.”

DIAPPOSITIVA 14 El resultado central

“El resultado central. Réplicas significa envasadoras trabajando en paralelo sobre la misma cola.

Con **una** —la línea sin escalar—, envasado acumula sesenta y ocho segundos de espera y el resto está casi ocioso. Con **dos**, el cuello **se desplaza a esterilización**. Y ojo: esterilización no cambió nada; lo que cambió es que ahora le llega trabajo más rápido de lo que puede sacar, porque envasado dejó de ser el freno. Con **tres**, envasado queda sobrado... y el cuello sigue en esterilización.

Ahí está la respuesta del caso: el cuello no se eliminó — se mudó.”

DIAPPOSITIVA 15 ¿Cuánto compra cada réplica?

“¿Cuánto compró la inversión? Para eso está el **lead time**: el tiempo de punta a punta, desde que se crea la orden hasta que sale terminada.

Frente a la línea sin escalar, la segunda envasadora lo reduce un veintiuno por ciento; la tercera, casi nada, porque el límite ya no está en envasado.

La conclusión útil: la siguiente inversión correcta es una segunda autoclave, no una tercera envasadora — y el tablero responde esa pregunta en vivo. Para eso sirve medir.”

DIAPPOSITIVA 16 Balanceo con una réplica lenta

“Este experimento prueba que el balanceo es dinámico de verdad: tres réplicas sobre la misma cola, una al doble de ciclo, treinta lotes.

Un round-robin habría dado diez, diez y diez, sin mirar la velocidad de nadie. Medimos once, doce y siete: la lenta procesó un cuarenta por ciento menos.

Nadie programó esa proporción: emergió de que cada réplica pide trabajo solo cuando queda libre. Balanceo proporcional a la capacidad real de cada máquina.”

DIAPPOSITIVA 17 Prueba de estrés: saturación

“La prueba de estrés: veinticinco órdenes de golpe, y después nadie toca nada.

Cómo leer el gráfico: cada fila es una etapa; cada celda, una consulta al diagnóstico —una cada quince segundos—; el color, el estado que respondió.

Qué nos dice: a los treinta y dos segundos el sistema declara crítico **por sí solo**. **⟨opcional⟩** La congestión avanza por la línea igual que el trabajo, como una ola. Y a los doscientos setenta y ocho, todo vuelve a normal **sin intervención**: las esperas viejas salieron de la ventana móvil.

El diagnóstico funciona solo en los dos sentidos: declara el problema cuando aparece y lo da de baja cuando pasa.”

DIAPPOSITIVA 18 Qué pasa cuando una pieza se cae

“¿Y si una pieza se cae? Lo probamos con la falla más grave: detener Redis, que es quien reparte el trabajo.

El antes: la API quedaba cuarenta y seis segundos muda. El diagnóstico: el cuelgue no estaba en la conexión sino en la **resolución del nombre** — Docker borra el nombre del contenedor detenido y la búsqueda tarda unos cuarenta y cinco segundos en rendirse, fuera del alcance del timeout que teníamos. La solución: un plazo máximo de dos segundos; si no hay respuesta, un error inmediato que **nombra al servicio caído**.

El después, medido: dos coma un segundos, con explicación.”

DIAPPOSITIVA 19 Persistencia de datos

“Todo lo que la línea hace queda registrado: cada acción de cada lote es un evento en SQLite, y las métricas se calculan siempre desde los eventos — nada se guarda dos veces, así ningún dato contradice a otro.

¿Por qué una base de datos y no una planilla o un archivo? Hasta seis contenedores escriben al mismo tiempo: SQLite ordena esas escrituras para que entren de a una sin pisarse. Un archivo compartido las perdería o mezclaría.

Y la reproducibilidad, probada: en un computador limpio, un solo comando levantó el sistema completo — los ocho programas arrancaron y confirmaron estar operativos.”

DIAPPOSITIVA 20 Límites conocidos y su solución

“Cierro la parte técnica con los límites que conocemos, porque saber qué falla primero también es diseño.

Si cae Redis, el reparto se detiene: la solución estándar es una copia de respaldo con conmutación automática. Si una estación muere a mitad de ciclo, ese lote se pierde: el camino es apartarlo a una cola ‘en proceso’ y devolverlo si la réplica deja de dar señales. SQLite atiende bien un nodo; con más escritores se migra a un motor con servidor tocando un solo módulo. Y la API y el tablero se duplican tras un balanceador.

Lo importante: cada mejora es local — se toca un componente sin rediseñar los demás, y para eso se dividió el sistema en nodos.”

DIAPPOSITIVA 21 Conclusiones

“Para cerrar, cinco frases, todas medidas.

El cuello no se elimina, se desplaza: de envasado a esterilización al pasar de una a dos envasadoras.

Medir espera es lo que permite localizarlo: el cuello real tenía un ciclo más corto que envasado.

El balanceo por demanda se adapta solo: once, doce y siete con una réplica lenta, sin que nadie asignara ese reparto.

La condición de carrera se elimina, no se administra: cero duplicados en doscientas órdenes.

Las fallas se explican, no se enmascaran: una respuesta clara en dos segundos, en vez de cuarenta y seis de silencio.

Si me quedo con una sola idea, profesor, es esta: agregar máquinas no elimina el cuello de botella — lo traslada. Por eso lo que hay que construir no es una línea más rápida, sino una línea que sepa decirte dónde se está atascando ahora mismo. Sin eso, se invierte a ciegas.”

DIAPPOSITIVA 22 Gracias

“Muchas gracias. Quedamos atentos a sus preguntas.”

Preguntas probables

¿Dónde está el balanceador?

Es la cola compartida: el punto único por el que pasa todo el trabajo y que decide quién procesa qué. Elegimos reparto por demanda porque un balanceador clásico no es dinámico — le sigue mandando trabajo a la réplica lenta. El experimento de la réplica lenta lo muestra adaptándose solo.

¿Y el lock o contador distribuido?

Lo cubre el mismo mecanismo. La extracción atómica de Redis es un candado implícito por elemento, porque Redis atiende sus comandos de uno en uno. Un candado explícito con expiración habría *administrado* la ventana de carrera; la extracción atómica la *elimina*. Y lo demostramos: dos pasos da duplicados, un paso da cero en doscientas.

¿Por qué el cuello es esterilización si envasado es más lento (12 s contra 7 s)?

Porque con dos réplicas el tiempo efectivo de envasado es de unos seis segundos por lote. Y en general el cuello no es la etapa de mayor ciclo sino la de mayor espera: la que recibe trabajo más rápido de lo que puede drenarlo.

¿Qué pasa si una réplica muere a mitad de un lote?

Ese lote ya salió de la cola y se pierde de la línea. Es una limitación conocida y aceptada, porque el caso excluía tolerancia a fallos. Sabemos el camino: mover el lote a una cola «en proceso» en vez de sacarlo, y un barrendero que lo devuelva al detectar que la réplica dejó de dar señales.

¿Por qué SQLite y no un motor con servidor?

Un solo nodo de estado, escrituras cortas, y cero infraestructura extra que administrar en la demo. Lo estresamos en el experimento de la réplica lenta y encontramos su límite, que está documentado. El cambio a otro motor toca un solo módulo si hiciera falta.

¿Cómo saben que las métricas son correctas?

Los tiempos de servicio que medimos coinciden con los ciclos configurados —cuatro coma ocho, doce coma cuatro, tres coma cuatro y siete coma tres, contra cuatro, doce, tres y siete—. La instrumentación se valida sola. Y todo se calcula desde la tabla de eventos, así que no hay un segundo lugar que pueda contradecirla.

¿Por qué la espera se mide contra la entrada a cada cola y no contra la creación de la orden?

Porque con cuatro etapas en serie, medir contra la creación le cargaría a sellado todo lo que pasó antes en fileteado y envasado, y señalaría al culpable equivocado. Cada etapa responde solo por su propia cola.